AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 22: Probabilistic Inference

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Generative models, Hierarchical Models, MAP, Empirical Bayes, Mixture of Gaussians, Expectation Maximisation

2 Main Points

2.1 Beyond Conjugate Priors

- In the lecture on Bayesian inference we chose priors so that the posterior had the same form as the prior (conjugate priors)
- Real problems involve many interacting random variables, some of which we never observe, and the posterior then has no simple closed form
- This lecture is about how to set up such models and about simple strategies for using them: MAP, empirical Bayes and the EM algorithm. The lecture on MCMC shows how to compute the full posterior
- Everything rests on the two rules from the lecture on probability
  - the sum rule (marginalisation): $\mathbb{P}[X] = \sum_y \mathbb{P}[X, Y = y]$, or $f(x) = \int f(x, y)\,\mathrm{d}y$
  - the product rule (the definition of conditional probability): $\mathbb{P}[X, Y] = \mathbb{P}[X|Y]\ \mathbb{P}[Y] = \mathbb{P}[Y|X]\ \mathbb{P}[X]$, of which Bayes' rule is a rearrangement
  - conditioning does not imply causation

2.2 Building Probabilistic Models

- **Discriminative Models**
  - Often in machine learning our goal is to learn the probability distribution $\mathbb{P}[Y|\boldsymbol{X}]$ where Y is our target and $\boldsymbol{X}$ is a data point
  - We may parameterise this distribution with some parameters $\boldsymbol{\Theta}$ and our task would be to learn these parameters based on training data
- **Generative Models**
  - Surprisingly it is often easier to model the joint probability $\mathbb{P}[Y, \boldsymbol{X}]$
  - This means that we model the process of both generating the targets and the feature vectors together
  - These are known as *generative models* as they allow us to generate random examples

- We don't necessarily want to use them to generate random samples; it just makes the modelling process easier (although you need to get used to this as it feels counter-intuitive)
- We can use generative models to do discrimination since $\mathbb{P}[Y|\boldsymbol{X}] = \mathbb{P}[Y, \boldsymbol{X}]/\mathbb{P}[\boldsymbol{X}]$ where $\mathbb{P}[\boldsymbol{X}] = \sum_Y \mathbb{P}[Y, \boldsymbol{X}]$ (the sum is over the few values of the target, not over all possible feature vectors)

- **Latent Variables**
  - To model complicated processes we introduce intermediate random variables that we never observe
  - These are known as **latent variables**; they are not our target, but they make the model easy to write down
  - To predict we marginalise them out: $\mathbb{P}[\boldsymbol{X}, Y] = \sum_Z \mathbb{P}[\boldsymbol{X}, Y, Z]$

- **Generative Stories**
  - A good way to build a model is to write down how the data could have been generated, as if writing a simulator. Each line draws one variable given variables drawn earlier. For the ratings model below

```
theta = rng.beta(a, b, size=J)    # a rate for each product
k = rng.binomial(n, theta)        # observed positive reviews
```

- By the product rule the joint distribution is the product of the distributions in each line

$$f(\boldsymbol{\theta}, \boldsymbol{k}) = \prod_{j=1}^{J} \mathrm{Beta}(\theta_j|a, b)\,\mathrm{Bin}(k_j|n_j, \theta_j)$$

- *Probabilistic programming languages* such as Stan and PyMC let you write a model in this form and then do the inference (usually by MCMC) for you

- **Graphical Models**
  - The same information can be drawn as a diagram (Figure 1): each random variable is a node, and arrows point to a variable from the variables it is drawn from
  - We use dark nodes for fixed numbers (parameters or known quantities), shaded nodes for observed variables and white nodes for latent variables
  - A box (a *plate*) means "repeat for each": here for each product j
  - The joint distribution is the product, over nodes, of the distribution of each node given its parents
  - Graphical models are also used to read off which variables are independent, but we only use them to design and explain models

2.3 Hierarchical Models

- A shop lists products with the fraction of reviews that are positive. Product A has 3 positive reviews out of 3 (100%), product B 1840 out of 2000 (92%). Which is better?

- Ranking by the fraction puts every product with a handful of good reviews at the top. Intuitively we trust A's 100% less, and our experience that most products are rated about 80% should pull it down

- **Model.** Product j has an unknown probability θ_j that a review is positive, and k_j of its n_j reviews are positive

$$k_j \sim \mathrm{Bin}(n_j, \theta_j), \qquad \theta_j \sim \mathrm{Beta}(a, b)$$

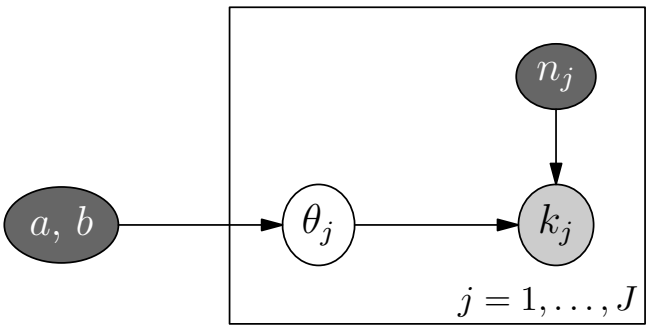


Figure 1: Graphical model for the hierarchical ratings model

- the products are different, but similar: their θ_j come from a common population
- a and b are *hyperparameters*: parameters of the distribution of the parameters. $a/(a+b)$ is the typical rate and $a+b$ measures how alike the products are (the variance of $\text{Beta}(a,b)$ is $ab/((a+b)^2(a+b+1))$, which is small when $a+b$ is large)
- the model has levels (population, product, reviews), so it is called a *hierarchical model*

- **One product at a time.** If we knew a and b, each product would be the beta-binomial problem of the lecture on Bayesian inference, with posterior

$$f(\theta_j|k_j) = \text{Beta}(\theta_j|a+k_j,\, b+n_j-k_j)$$

The posterior mean is a weighted average of the typical rate and the observed fraction

$$\mathbb{E}[\theta_j|k_j] = \frac{a+k_j}{a+b+n_j} = \frac{a+b}{a+b+n_j}\,\frac{a}{a+b} + \frac{n_j}{a+b+n_j}\,\frac{k_j}{n_j}$$

so the prior counts as $a+b$ extra reviews. With $a=20$ and $b=5$ (typical rate 0.8, worth 25 reviews) product A gets $23/28 \approx 0.82$ and product B gets $1860/2025 \approx 0.92$: B is better

- **Shrinkage.** Products with few reviews are pulled (*shrunk*) towards the typical rate, while those with many reviews barely move (Figure 2). Each product *borrows strength* from the others

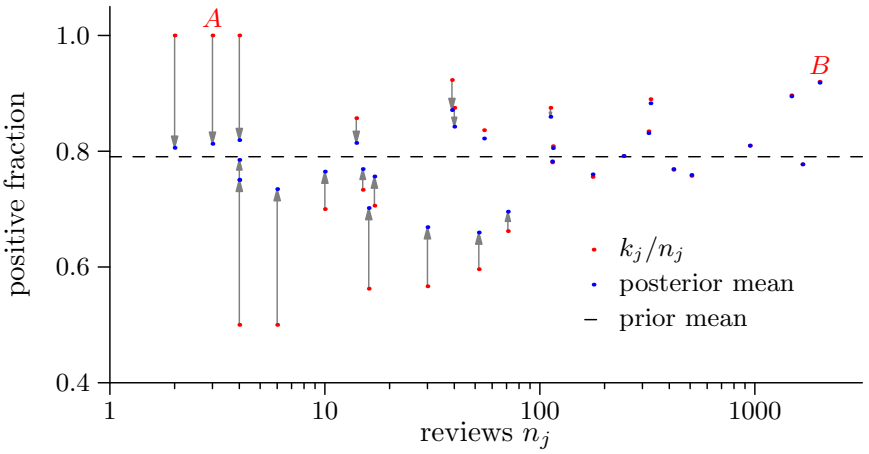


Figure 2: Fraction of positive reviews (red) and posterior means (blue) for 30 products; A and B are the examples above. The hyperparameters were fitted by empirical Bayes

- **Partial pooling.** There are two extremes: estimate each product on its own (k_j/n_j: no pooling), which is noisy when n_j is small, or treat all products as the same (complete pooling), which ignores real differences. A hierarchical model sits in between, and the data

decide where, through $a + b$. The same idea is used for schools, hospitals, adverts, sports players and the users of a recommender system

- But we don't know a and b. If we give them a prior too (a *hyperprior*), the posterior $f(a, b, \boldsymbol{\theta}|\boldsymbol{k})$ is no longer conjugate and has no closed form: a and b appear inside the beta function that normalises $\mathrm{Beta}(\theta_j|a, b)$

2.4 Solution Strategies

- **Full Bayes.** Bayes' rule gives the posterior over all the parameters

$$\mathbb{P}[\boldsymbol{\Theta}|\mathcal{D}] = \frac{\mathbb{P}[\mathcal{D}|\boldsymbol{\Theta}]\ \mathbb{P}[\boldsymbol{\Theta}]}{\mathbb{P}[\mathcal{D}]}$$

  - it describes our uncertainty and, because it averages over parameters rather than choosing the best, it is much less prone to over-fitting
  - but it is difficult: the posterior is rarely a standard distribution, and the *evidence* or *marginal likelihood*

$$\mathbb{P}[\mathcal{D}] = \sum_{\boldsymbol{\Theta}} \mathbb{P}[\mathcal{D}|\boldsymbol{\Theta}]\ \mathbb{P}[\boldsymbol{\Theta}] \qquad \text{or} \qquad f(\mathcal{D}) = \int f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}$$

    is a sum or integral over far too many values when $\boldsymbol{\Theta}$ is high dimensional
  - Monte Carlo methods (the lecture on MCMC) can sample from the posterior without computing the evidence, but they can be slow

- **Maximum a posteriori (MAP).** Take the mode of the posterior

$$\boldsymbol{\theta}_{\mathrm{MAP}} = \operatorname*{argmax}_{\boldsymbol{\theta}} f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta}) = \operatorname*{argmax}_{\boldsymbol{\theta}} \log(f(\mathcal{D}|\boldsymbol{\theta})) + \log(f(\boldsymbol{\theta}))$$

  - we don't need the evidence, which does not depend on $\boldsymbol{\theta}$
  - $-\log(f(\boldsymbol{\theta}))$ acts as a regulariser. For example a Gaussian prior $w_i \sim \mathcal{N}\big(w_i\big|0, s^2\big)$ on weights adds $\|\boldsymbol{w}\|^2/(2s^2)$ to the loss: ridge regression, or weight decay in neural networks. We use this in the lecture on matrix factorisation
  - but it isn't really Bayesian (although some users will claim it is). It keeps a single point and throws away our uncertainty about $\boldsymbol{\theta}$, so we lose our error bars and can over-fit
  - the peak of a probability density is not invariant: change variables (e.g. from σ to $\log \sigma$) and the peak moves, although the probabilities do not
  - in high dimensions most of the posterior probability can lie far from the peak, so the MAP point can be untypical

- **Maximum likelihood.** With a uniform prior MAP is the *maximum likelihood estimate*

  - this was developed and popularised by Ronald Fisher (1912–1922) at a time when Bayesian inference was considered taboo, and was accepted by the statistical community despite its strong connection to Bayesian inference
  - in high-dimensional problems a non-uniform prior can make a big difference

- **Empirical Bayes.** For a hierarchical model a half-way house is to integrate out the many parameters θ_j, choose the few hyperparameters to maximise the evidence (as we did for the hyperparameters of a Gaussian process), and then use the conjugate posterior for each product

- for the ratings model the integral is in closed form, the *beta-binomial* distribution

$$\mathbb{P}[k_j|a,b] = \int_0^1 \binom{n_j}{k_j}\theta^{k_j}(1-\theta)^{n_j-k_j}\frac{\theta^{a-1}(1-\theta)^{b-1}}{B(a,b)}\,\mathrm{d}\theta = \binom{n_j}{k_j}\frac{B(a+k_j,\, b+n_j-k_j)}{B(a,b)}$$

  where $B(a,b) = \int_0^1 \theta^{a-1}(1-\theta)^{b-1}\mathrm{d}\theta = \Gamma(a)\Gamma(b)/\Gamma(a+b)$ is the beta function
- we maximise $\sum_j \log \mathbb{P}[k_j|a,b]$ numerically; for the 30 products of Figure 2 this gives $a = 19.9$ and $b = 5.3$
- it is cheap and often works well, but it ignores our uncertainty in a and b

- **A ladder of strategies**

Strategy	What you get	Cost
Maximum likelihood	a point estimate	an optimiser
MAP	a regularised point estimate	an optimiser
Empirical Bayes	posteriors given fitted hyperparameters	an integral
EM	ML or MAP with latent variables	iteration
Full Bayes	all the uncertainty	MCMC

  Choose the cheapest strategy that answers your question

2.5 You Already Know EM

- k-means clustering repeats two steps: assign each point to its nearest centre, then move each centre to the mean of its points
- The first step guesses the hidden cluster labels; the second fits the parameters as if those guesses were right
- The EM algorithm does the same, but with *probabilities* of belonging to each cluster, and it comes from a probabilistic model, a mixture

2.6 Mixtures of Gaussians

- To illustrate latent variables and a simple hierarchical model we consider a classic probabilistic model known as a *mixture of Gaussians*
- We consider a concrete scenario
- We suppose we are detecting two types (A and B) of short-lived particles [1]
- We measure the mass, X_i, of each particle with a noisy detector, but we don't know the type of particle
- Let $Z_i \in \{0,1\}$ equal 1 if particle i is of type A and 0 if it is of type B, with $\mathbb{P}[Z_i = 1] = p$
- The probability density of the measurement is therefore

$$f(X_i|Z_i,\boldsymbol{\Theta}) = Z_i\,\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big) + (1-Z_i)\,\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)$$

  - where μ_A and μ_B are the masses of particles of type A and B respectively
  - σ_A and σ_B are the standard deviations in the measurements
  - this says that if the ith particle is of type A then the density of X_i is $\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big)$, while if it is of type B, then X_i is distributed according to $\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)$
- We show some typical data from $m = 1\,000$ observations in Figure 3

[1] If you prefer, you can think of an autonomous vehicle using lidar where it detects reflections from two different, but close by, objects, A and B. We make multiple noisy measurements of the distance from the two objects.

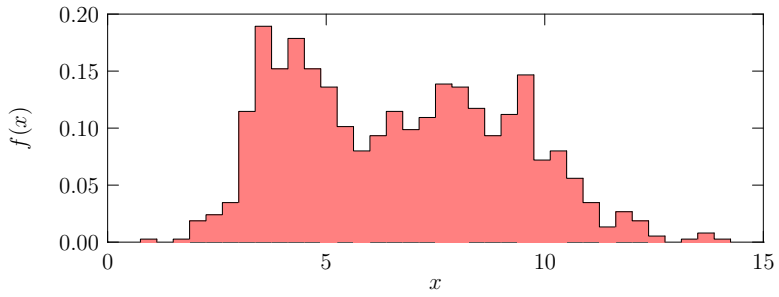


Figure 3: Example of data measuring the masses of two types of particles

- Our job is to infer the parameters $\boldsymbol{\Theta} = (\mu_A, \sigma_A^2, \mu_B, \sigma_B^2, p)$
- As a graphical model (Figure 4): for each data point draw a type Z_i, then a mass X_i from that type's normal distribution. Z_i is latent; the parameters are shared by all the data points, so they sit outside the plate

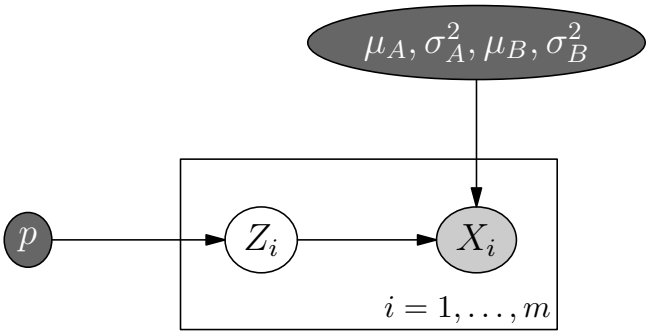


Figure 4: Graphical model for a mixture of two Gaussians

- We could do a full Bayesian calculation, but let us just use maximum likelihood
- Assuming the data points are independent, the likelihood of the data $\mathcal{D} = \{X_i | i = 1, 2, \ldots, m\}$ is

$$
\begin{aligned}
f(\mathcal{D}|\boldsymbol{\Theta}) &\overset{(1)}{=} \sum_{\boldsymbol{Z} \in \{0,1\}^m} f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}) \\
&\overset{(2)}{=} \prod_{i=1}^{m} \sum_{Z_i \in \{0,1\}} f(X_i, Z_i|\boldsymbol{\Theta}) \overset{(3)}{=} \prod_{i=1}^{m} \sum_{Z_i \in \{0,1\}} f(X_i|Z_i, \boldsymbol{\Theta})\, \mathbb{P}[Z_i|\boldsymbol{\Theta}]
\end{aligned}
$$

  1. where we marginalise out the latent variables $\boldsymbol{Z} = (Z_1, Z_2, \ldots Z_m)$
  2. we assume the data points are independent, so the sum over $\boldsymbol{Z}$ factorises into a product of sums over each Z_i
  3. we use the identity $f(X_i, Z_i|\boldsymbol{\Theta}) = f(X_i|Z_i, \boldsymbol{\Theta})\, \mathbb{P}[Z_i|\boldsymbol{\Theta}]$

- It is usually easier working with the log-likelihood

$$
\begin{aligned}
\log(f(\mathcal{D}|\boldsymbol{\Theta})) &= \sum_{i=1}^{m} \log\big(f(X_i|Z_i = 1)\, \mathbb{P}[Z_i = 1] + f(X_i|Z_i = 0)\, \mathbb{P}[Z_i = 0]\big) \\
&= \sum_{i=1}^{m} \log\big(p\, \mathcal{N}\big(X_i \big| \mu_A, \sigma_A^2\big) + (1 - p)\, \mathcal{N}\big(X_i \big| \mu_B, \sigma_B^2\big)\big)
\end{aligned}
$$

- We could do gradient ascent on this, but it is an ugly expression to work with: the log of a sum does not simplify

- Strictly, the likelihood has no maximum: put $\mu_A = X_1$ and let $\sigma_A \to 0$. Then $f(X_1|\boldsymbol{\Theta}) \geq p\mathcal{N}\big(X_1\big|X_1, \sigma_A^2\big) = p/(\sqrt{2\pi}\,\sigma_A) \to \infty$, while every other data point still has a density of at least $(1-p)\mathcal{N}\big(X_i\big|\mu_B, \sigma_B^2\big) > 0$. What we want is a good *local* maximum

2.7 Expectation Maximisation

- Rather than maximise the likelihood directly we can start from some initial guess $\boldsymbol{\Theta}^{(0)}$ and iteratively improve it

$$\boldsymbol{\Theta}^{(t+1)} = \operatorname*{argmax}_{\boldsymbol{\Theta}} Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}), \qquad Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = \sum_{\boldsymbol{Z}\in\{0,1\}^m} \mathbb{P}\Big[\boldsymbol{Z}\Big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\Big]\, \log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})) \tag{1}$$

  - $\log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}))$ is the log-likelihood we would have if we knew the latent variables (the *complete-data* log-likelihood); it is usually simple
  - we don't know $\boldsymbol{Z}$, so we average over it using its distribution given the data and our current parameters

- This is a general optimisation strategy that is regularly used when we have latent variables
- It is known as **expectation maximisation** or the **EM-algorithm**
  - E-step: compute $\mathbb{P}\big[\boldsymbol{Z}\big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$
  - M-step: maximise $Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$
- This looks very different to maximising the log-likelihood: it takes some effort to understand why this works
- By the definition of conditional probability

$$f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}) = \mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]\; f(\mathcal{D}|\boldsymbol{\Theta})$$

  from which (taking logs and rearranging), for every $\boldsymbol{Z}$,

$$\log(f(\mathcal{D}|\boldsymbol{\Theta})) = \log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})) - \log(\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}])$$

- Multiply both sides by $\mathbb{P}\big[\boldsymbol{Z}\big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$ and sum over $\boldsymbol{Z}$. The left-hand side does not involve $\boldsymbol{Z}$, and the probabilities sum to 1, so

$$\log(f(\mathcal{D}|\boldsymbol{\Theta})) = Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) + S(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$$

  where Q is the function we maximise in equation (1) and

$$S(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = -\sum_{\boldsymbol{Z}\in\{0,1\}^m} \mathbb{P}\Big[\boldsymbol{Z}\Big|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\Big]\, \log(\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}])$$

- The change in log-likelihood when we update our parameters is therefore

$$\begin{aligned}\Delta L &= \log\Big(f(\mathcal{D}|\boldsymbol{\Theta}^{(t+1)})\Big) - \log\Big(f(\mathcal{D}|\boldsymbol{\Theta}^{(t)})\Big) \\ &= Q(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) - Q(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)}) + S(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) - S(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)})\end{aligned}$$

- The difference of the S terms is

$$\sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log\left(\frac{P_t(\boldsymbol{Z})}{P_{t+1}(\boldsymbol{Z})}\right) = \mathrm{KL}(P_t \| P_{t+1})$$

  where $P_t(\boldsymbol{Z}) = \mathbb{P}\big[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$; this is the *KL divergence,* which we meet again in the lecture on entropy

- KL divergences are never negative. Using $\log(x) \le x - 1$,

$$-\mathrm{KL}(P_t \| P_{t+1}) = \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \log\left(\frac{P_{t+1}(\boldsymbol{Z})}{P_t(\boldsymbol{Z})}\right) \le \sum_{\boldsymbol{Z}} P_t(\boldsymbol{Z}) \left(\frac{P_{t+1}(\boldsymbol{Z})}{P_t(\boldsymbol{Z})} - 1\right) = 1 - 1 = 0$$

- In expectation maximisation we choose $\boldsymbol{\Theta}^{(t+1)}$ to maximise $Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$, so $Q(\boldsymbol{\Theta}^{(t+1)}|\boldsymbol{\Theta}^{(t)}) \ge Q(\boldsymbol{\Theta}^{(t)}|\boldsymbol{\Theta}^{(t)})$
- Thus $\Delta L \ge 0$: no step decreases the log-likelihood. (We only need Q to increase, so a partial M-step also works.)
- To find a MAP solution instead, add $\log(f(\boldsymbol{\Theta}))$ to Q
- Let us return to our two types of particles A and B
- The complete-data likelihood is $f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}) = \prod_{i=1}^{m} f(X_i, Z_i|\boldsymbol{\Theta})$ where

$$f(X_i, Z_i|\boldsymbol{\Theta}) = p\, Z_i\, \mathcal{N}\big(X_i \big| \mu_A, \sigma_A^2\big) + (1-p)\,(1-Z_i)\,\mathcal{N}\big(X_i \big| \mu_B, \sigma_B^2\big)$$

- E-step. Given the data, the Z_i are independent and by Bayes' rule

$$\gamma_i^{(t)} = \mathbb{P}\left[Z_i = 1 \middle| X_i, \boldsymbol{\Theta}^{(t)}\right] = \frac{p^{(t)}\,\mathcal{N}\left(X_i \middle| \mu_A^{(t)}, \sigma_A^{2(t)}\right)}{p^{(t)}\,\mathcal{N}\left(X_i \middle| \mu_A^{(t)}, \sigma_A^{2(t)}\right) + (1-p^{(t)})\,\mathcal{N}\left(X_i \middle| \mu_B^{(t)}, \sigma_B^{2(t)}\right)}$$

  and $\mathbb{P}\left[Z_i = 0 \middle| X_i, \boldsymbol{\Theta}^{(t)}\right] = 1 - \gamma_i^{(t)}$. $\gamma_i^{(t)}$ is called the *responsibility* of type A for data point i
- M-step. Because $\log(f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta}))$ is a sum over data points, the sum over $\boldsymbol{Z}$ in Q reduces to a sum over each Z_i

$$\begin{aligned} Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) &= \sum_{i=1}^{m} \gamma_i^{(t)} \log\big(p\,\mathcal{N}\big(X_i \big| \mu_A, \sigma_A^2\big)\big) + (1-\gamma_i^{(t)}) \log\big((1-p)\,\mathcal{N}\big(X_i \big| \mu_B, \sigma_B^2\big)\big) \\ &= \sum_{i=1}^{m} \gamma_i^{(t)} \left(\log(p) - \frac{(X_i - \mu_A)^2}{2\sigma_A^2} - \frac{1}{2}\log\big(2\,\pi\,\sigma_A^2\big)\right) \\ &\qquad + (1-\gamma_i^{(t)}) \left(\log(1-p) - \frac{(X_i - \mu_B)^2}{2\sigma_B^2} - \frac{1}{2}\log\big(2\,\pi\,\sigma_B^2\big)\right) \end{aligned}$$

- To maximise this we just set the derivatives to 0
  - With respect to p

$$\frac{\partial Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})}{\partial p} = \frac{1}{p}\sum_{i=1}^{m} \gamma_i^{(t)} - \frac{1}{1-p}\sum_{i=1}^{m} (1-\gamma_i^{(t)}) = 0 \quad \Rightarrow \quad p^{(t+1)} = \frac{1}{m}\sum_{i=1}^{m} \gamma_i^{(t)}$$

  - With respect to μ_A

$$\frac{\partial Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})}{\partial \mu_A} = \sum_{i=1}^{m} \gamma_i^{(t)} \frac{X_i - \mu_A}{\sigma_A^2} = 0$$

    so (and similarly for μ_B)

$$\mu_A^{(t+1)} = \frac{\sum_{i=1}^{m} \gamma_i^{(t)} X_i}{\sum_{i=1}^{m} \gamma_i^{(t)}}, \qquad \mu_B^{(t+1)} = \frac{\sum_{i=1}^{m} (1-\gamma_i^{(t)}) X_i}{\sum_{i=1}^{m} (1-\gamma_i^{(t)})}$$

- With respect to σ_A^2

$$\frac{\partial Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})}{\partial \sigma_A^2} = \frac{1}{2\,\sigma_A^4}\sum_{i=1}^{m}\gamma_i^{(t)}(X_i-\mu_A)^2 - \frac{1}{2\,\sigma_A^2}\sum_{i=1}^{m}\gamma_i^{(t)} = 0$$

so, putting in the optimal $\mu_A^{(t+1)}$ (and similarly for σ_B^2)

$$\sigma_A^{2(t+1)} = \frac{\sum_{i=1}^{m}\gamma_i^{(t)}(X_i-\mu_A^{(t+1)})^2}{\sum_{i=1}^{m}\gamma_i^{(t)}}, \qquad \sigma_B^{2(t+1)} = \frac{\sum_{i=1}^{m}(1-\gamma_i^{(t)})(X_i-\mu_B^{(t+1)})^2}{\sum_{i=1}^{m}(1-\gamma_i^{(t)})}$$

- These are very natural update equations
  - we estimate the probability, $\gamma_i^{(t)}$, that observation X_i is a particle of type A based on our current parameters
  - we then compute the fraction, means and variances of each type, weighting each data point by these probabilities
  - if the $\gamma_i^{(t)}$ were all 0 or 1 these would be the ordinary fraction, means and variances of each group
- We are guaranteed that our EM-algorithm never decreases the likelihood (although it could reach a local rather than global optimum)
- For the data set we showed earlier (which was a random sample of size 1000 generated using $p = 0.3$, $\mu_A = 4$, $\sigma_A = 0.8$, $\mu_B = 8$ and $\sigma_B = 2$) we get the results shown in Figure 5

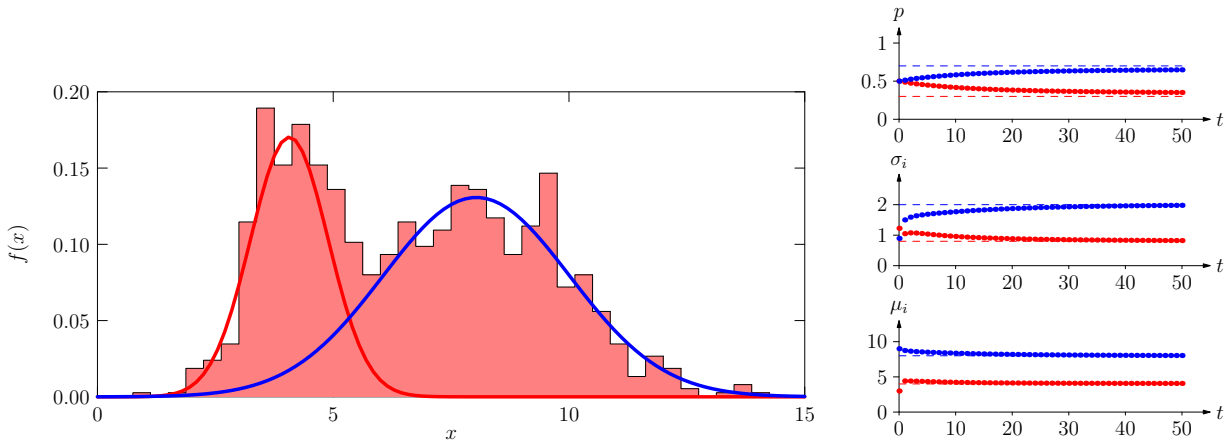


Figure 5: Example of EM algorithm to compute the statistics for the masses of our two particles

- The EM algorithm often leads to very natural update equations, but its convergence is often rather slow

2.8 EM and k-Means

- Give both clusters the same fixed variance σ^2. Then

$$\gamma_i = \frac{1}{1+\frac{1-p}{p}\,\mathrm{e}^{((X_i-\mu_A)^2-(X_i-\mu_B)^2)/(2\sigma^2)}}$$

and as $\sigma \to 0$ (with $0 < p < 1$ fixed) $\gamma_i \to 1$ if X_i is nearer μ_A and $\gamma_i \to 0$ if it is nearer μ_B

- The E-step becomes "assign each point to its nearest mean" and the M-step "move each mean to the average of its points": k-means is EM with hard assignments
- EM's soft assignments let a point in the overlap count for both clusters, and EM also learns the size (p) and width (σ) of each cluster

2.9 EM in Practice

- EM finds a local maximum that depends on the starting point, so run it from several starting points and keep the best
- A component can collapse onto a single point ($\sigma \to 0$), where the likelihood is infinite. A prior on σ (i.e. MAP, which just adds $\log(f(\boldsymbol{\Theta}))$ to Q) prevents this
- Convergence can be slow when the components overlap, because the latent variables are then very uncertain
- The update equations are simple and need no step size
- The same idea applies to any model with latent variables: missing data, hidden Markov models, and many more

2.10 Lower Bounds and VAEs

- (This section goes beyond the lecture: it is a pointer to where these ideas lead.)
- The proof that EM works can be written for *any* distribution $q(\boldsymbol{Z})$ over the latent variables. Since $f(\mathcal{D}|\boldsymbol{\Theta}) = f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})/\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]$ for every $\boldsymbol{Z}$,

$$\begin{aligned}\log(f(\mathcal{D}|\boldsymbol{\Theta})) &= \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \log\left(\frac{f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})}{\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]}\right) \\ &= \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \log\left(\frac{f(\mathcal{D}, \boldsymbol{Z}|\boldsymbol{\Theta})}{q(\boldsymbol{Z})}\right) + \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \log\left(\frac{q(\boldsymbol{Z})}{\mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]}\right)\end{aligned}$$

  The second term is $\mathrm{KL}\big(q \,\|\, \mathbb{P}[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}]\big) \geq 0$, so the first term is a lower bound on the log-likelihood, called the *evidence lower bound* (ELBO)
- EM is coordinate ascent on the ELBO: the E-step sets q to the posterior $\mathbb{P}\big[\boldsymbol{Z}|\mathcal{D}, \boldsymbol{\Theta}^{(t)}\big]$, which makes the KL term zero so that the bound touches the log-likelihood; the M-step maximises the bound over $\boldsymbol{\Theta}$ (the $-\sum q \log q$ part does not depend on $\boldsymbol{\Theta}$, so this is maximising Q)
- A *variational autoencoder* (VAE) is a latent-variable model in which a neural network (the decoder) generates data from a latent vector. Its posterior over the latent vector is intractable, so a second network (the encoder) learns an approximation q, and both are trained by gradient ascent on the same bound
- We meet the KL divergence again in the lecture on entropy

3 Exercises

3.1 Posterior means for ratings

- With $a = 20$ and $b = 5$, product C has 9 positive reviews out of 10 and product D 85 out of 100. Rank them by the fraction of positive reviews and by the posterior mean.
- Show that the posterior mean $(a + k)/(a + b + n)$ is the weighted average of $a/(a + b)$ and k/n given in the notes.

3.2 The beta-binomial

- Derive the marginal likelihood $\mathbb{P}[k|a, b]$ of the ratings model, using $\int_0^1 \theta^{\alpha-1}(1-\theta)^{\beta-1}\mathrm{d}\theta = B(\alpha, \beta)$. Why can the factor $\binom{n}{k}$ be ignored when choosing a and b?

3.3 Generative to discriminative

- In a mixture of two Gaussians with $p = 0.3$, $\mu_A = 4$, $\sigma_A = 0.8$, $\mu_B = 8$ and $\sigma_B = 2$, a particle of mass $X = 5.5$ is measured. What is the probability that it is of type A?

3.4 One EM step by hand

- Data $X = (0, 1, 5, 6)$; current parameters $p = \frac{1}{2}$, $\mu_A = 1$, $\mu_B = 5$, $\sigma_A = \sigma_B = 1$ (keep the variances fixed). Compute the responsibilities and the new p, μ_A and μ_B.

3.5 Mysterious Disease

- We assume that we are tracking some disease
- Let $Z(t)$ be the number of people that catch the disease on day t, but this is unknown (a latent variable)
- We assume the rate of growth of the disease is

$$\mathbb{P}[Z(t+1)] = \text{Poi}\Big(Z(t+1)\Big|\frac{r_0}{3}\left(Z(t) + Z(t-1) + Z(t-2)\right)\Big)$$

  - We are assuming that someone is infectious for the first three days after catching the disease
  - We assume $Z(1) = 1$ and $Z(0) = Z(-1) = 0$
  - In expectation everyone with the disease will infect r_0 new people
- We observe $X(t)$, the number of new cases that are reported, where

$$\mathbb{P}[X(t) = k] = \text{Binom}(k|Z(t), p) = \binom{Z(t)}{k} p^k \left(1-p\right)^{Z(t)-k}$$

  - We assume that each patient will be tested with a probability p
  - We are given a time series $(X(1), X(2), \ldots, X(T))$
- Build a probabilistic model to estimate p and r_0
- See answers

4 Experiments

4.1 Empirical Bayes for ratings

- Simulate 200 products from the ratings model, fit a and b by maximising the evidence, and compare the posterior means with the raw fractions as estimates of the true θ_j

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import betaln

rng = np.random.default_rng(3)
J = 200
n = np.round(np.exp(rng.uniform(np.log(2), np.log(2000), J))).astype(int)
theta = rng.beta(16, 4, J)                    # true a = 16, b = 4
k = rng.binomial(n, theta)
```

```
def neg_log_evidence(log_ab):                # binomial coefficients omitted:
    a, b = np.exp(log_ab)                    # they do not depend on a, b
    return -np.sum(betaln(a + k, b + n - k) - betaln(a, b))


a, b = np.exp(minimize(neg_log_evidence, np.log([1.0, 1.0]),
                       method="Nelder-Mead").x)
post = (a + k) / (a + b + n)
print("a, b:", a, b)
small = n < 10
for name, est in [("k/n", k / n), ("posterior mean", post)]:
    print(name, "RMS error, all:", np.sqrt(np.mean((est - theta) ** 2)),
          " n<10:", np.sqrt(np.mean((est - theta)[small] ** 2)))
```

- Which estimate is better, and for which products does it matter most? (See the answers.)

4.2 EM for a mixture of Gaussians

- Generate the data of the lecture and run EM, printing the log-likelihood as we go

```
import numpy as np

rng = np.random.default_rng(0)
m = 1000
p, muA, sA, muB, sB = 0.3, 4.0, 0.8, 8.0, 2.0      # true parameters
Z = rng.random(m) < p                                # latent types
X = np.where(Z, rng.normal(muA, sA, m), rng.normal(muB, sB, m))


def normal(x, mu, var):
    return np.exp(-(x - mu) ** 2 / (2 * var)) / np.sqrt(2 * np.pi * var)


th = dict(p=0.5, muA=2.0, vA=1.0, muB=10.0, vB=1.0)   # initial guess
for t in range(51):
    a = th["p"] * normal(X, th["muA"], th["vA"])
    b = (1 - th["p"]) * normal(X, th["muB"], th["vB"])
    loglik = np.sum(np.log(a + b))
    if t in (0, 1, 2, 5, 10, 20, 50):
        print(f"t={t:2d}  log-lik {loglik:9.2f}  p={th['p']:.3f}  "
              f"muA={th['muA']:.2f}  sA={np.sqrt(th['vA']):.2f}  "
              f"muB={th['muB']:.2f}  sB={np.sqrt(th['vB']):.2f}")
    g = a / (a + b)                                  # E-step: responsibilities
    th["p"] = g.mean()                               # M-step
    th["muA"] = np.sum(g * X) / np.sum(g)
    th["muB"] = np.sum((1 - g) * X) / np.sum(1 - g)
    th["vA"] = np.sum(g * (X - th["muA"]) ** 2) / np.sum(g)
    th["vB"] = np.sum((1 - g) * (X - th["muB"]) ** 2) / np.sum(1 - g)
```

- Does the log-likelihood ever decrease? How quickly do the parameters settle down? Try other starting points. (See the answers.)

4.3 Mysterious Disease

- Build a simulator of your model (assume you know p and r_0)

- If you are feeling very adventurous you could try to solve your model to predict p and r_0, but be warned this is hard (you probably need to use MCMC, which is the subject of a later lecture)

```
import numpy as np

rng = np.random.default_rng(1)
r0, p, T = 2.0, 0.2, 30
Z = np.zeros(T, dtype=int)          # Z[t]: new infections on day t+1
Z[0] = 1
for t in range(1, T):
    Z[t] = rng.poisson(r0 / 3 * Z[max(t - 3, 0):t].sum())
X = rng.binomial(Z, p)              # each new case is reported with probability p
print("Z:", Z)
print("X:", X)
```

5 Answers

5.1 Posterior means for ratings

- By fraction C (0.90) beats D (0.85). By posterior mean C gets $29/35 \approx 0.83$ and D gets $105/125 = 0.84$, so D ranks higher: C's ten reviews are not enough to be sure it is better than a typical product
- $\frac{a+b}{a+b+n} \cdot \frac{a}{a+b} + \frac{n}{a+b+n} \cdot \frac{k}{n} = \frac{a}{a+b+n} + \frac{k}{a+b+n} = \frac{a+k}{a+b+n}$

5.2 The beta-binomial

- Collecting the powers of θ,

$$\begin{aligned}\mathbb{P}[k|a,b] &= \int_0^1 \binom{n}{k}\theta^k(1-\theta)^{n-k}\,\frac{\theta^{a-1}(1-\theta)^{b-1}}{B(a,b)}\,\mathrm{d}\theta = \frac{\binom{n}{k}}{B(a,b)}\int_0^1 \theta^{a+k-1}(1-\theta)^{b+n-k-1}\mathrm{d}\theta \\ &= \binom{n}{k}\frac{B(a+k,\,b+n-k)}{B(a,b)}\end{aligned}$$

- $\binom{n}{k}$ does not depend on a or b, so it only adds a constant to the log-evidence and does not change where its maximum is

5.3 Empirical Bayes for ratings

- The fit gives $a = 19.6$ and $b = 4.8$ (the true values are 16 and 4; the typical rates, 0.80 and 0.80, agree well)
- The root-mean-square error of the estimates of θ_j falls from 0.116 for the raw fractions to 0.050 for the posterior means. For the products with fewer than 10 reviews it falls from 0.236 to 0.073: shrinkage helps most where the data are few

5.4 Generative to discriminative

- By Bayes' rule

$$\mathbb{P}[Z=1|X] = \frac{0.3\,\mathcal{N}\big(5.5\big|4, 0.64\big)}{0.3\,\mathcal{N}\big(5.5\big|4, 0.64\big) + 0.7\,\mathcal{N}\big(5.5\big|8, 4\big)} = \frac{0.3 \times 0.0860}{0.3 \times 0.0860 + 0.7 \times 0.0913} \approx 0.29$$

- Although 5.5 is closer to μ_A than to μ_B, type B is both more common and much more spread out, so it is the more likely explanation

5.5 One EM step by hand

- $\mathcal{N}\big(x\big|\mu,1\big) \propto \mathrm{e}^{-(x-\mu)^2/2}$ and $p=\frac{1}{2}$, so $\gamma_i = 1/(1+\mathrm{e}^{-((X_i-\mu_B)^2-(X_i-\mu_A)^2)/2})$. For $X=0,1,5,6$ the exponents $(X_i-\mu_B)^2-(X_i-\mu_A)^2$ are $24, 16, -16, -24$, so $\gamma \approx (1.000, 1.000, 0.000, 0.000)$ (more precisely $1-6\times10^{-6}$, $1-3.4\times10^{-4}$, 3.4×10^{-4}, 6×10^{-6})
- $p=\frac{1}{4}\sum\gamma_i = 0.5$, $\mu_A \approx (0+1)/2 = 0.50$ and $\mu_B \approx (5+6)/2 = 5.50$: the clusters are so well separated that EM reduces to splitting the data into two groups and averaging each

5.6 EM for a mixture of Gaussians

- Running the code gives

t	log-likelihood	p	μ_A	σ_A	μ_B	σ_B
0	-4316.14	0.500	2.00	1.00	10.00	1.00
1	-2264.00	0.396	4.33	0.93	8.44	1.59
5	-2250.37	0.350	4.29	0.97	8.18	1.81
20	-2244.50	0.282	4.08	0.82	7.89	1.96
50	-2243.99	0.257	4.02	0.77	7.78	2.03

- The log-likelihood never decreases. The first step does almost all the work, but the parameters then creep for tens of iterations: by 100 they have settled at $p=0.255$, $\mu_A=4.01$, $\sigma_A=0.76$, $\mu_B=7.77$ and $\sigma_B=2.03$
- These are the maximum-likelihood estimates for this sample, not the true values; with 1000 points p is uncertain by a few hundredths, because the two groups overlap

5.7 Mysterious Disease

- We want to compute $f(p, r_0|\mathcal{D})$ where $\mathcal{D} = (X(1), X(2), \ldots, X(T))$
- We have a likelihood of

$$\mathbb{P}[X(t)|Z(t), p] = \mathrm{Binom}(X(t)|Z(t), p)$$

  where

$$\mathbb{P}[Z(t+1)|r_0, Z(t), Z(t-1), Z(t-2)] = \mathrm{Poi}\Big(Z(t+1)\Big|\frac{r_0}{3}\left(Z(t)+Z(t-1)+Z(t-2)\right)\Big)$$

- To perform a Bayesian calculation we would have to put priors, $f(p)$ and $f(r_0)$, on p and r_0
  - A reasonable prior to use for p is $f(p) = \mathrm{Beta}(p|a,b)$—you could use $a=b=1$ as an uninformative (uniform) prior, but you might have some prior knowledge, e.g. $a=b=2$ which says $f(p) = 6\,p\,(1-p)$
  - A reasonable prior for r_0 would be $\mathrm{Gamma}(r_0|a,b)$—you could use $a=b=0$ as an uninformative (improper) prior but again you might have some prior belief, e.g. $a=2$, $b=1$
- Bayes' rule tells us

$$f(p, r_0, \boldsymbol{Z}|\boldsymbol{X}) = \frac{\prod_{t=1}^{T} \mathbb{P}[X(t)|Z(t),p]\ \mathbb{P}[Z(t)|Z(t-1),Z(t-2),Z(t-3),r_0]\ f(r_0)\,f(p)}{\mathbb{P}[\boldsymbol{X}]}$$

- To get estimates of p and r_0 we have to marginalise out $\boldsymbol{Z}$
- Now the problem is this is rather horrible to compute (your priors are not conjugate priors in this problem and the posterior is very complicated)

- Probably the best way to do this is to use Markov Chain Monte Carlo (MCMC), which is the subject of a later lecture
- The simulator above gives, for $r_0 = 2$ and $p = 0.2$, case numbers that grow by roughly a factor of 1.45 a day; the reported numbers $X(t)$ are about a fifth of $Z(t)$, but noisy while they are small